

Tabibak — Local to Global Deployment Report

Transforming a local HTML app into a globally deployed, real-time synchronized web application

Live URL: <https://truthful-enchancement-production-2411.up.railway.app/tabibak.html>

GitHub: <https://github.com/hashemelhelo2827/tabibak>

1. Original State — Local Only

The application was a single `tabibak.html` file (~4300 lines) with embedded CSS and JavaScript. All data was stored in the browser's `localStorage` :

Feature	Original Implementation
Data Storage	<code>localStorage</code> (browser-only, single device)
User Auth	Plain text passwords in <code>localStorage</code> , no hashing
API	No backend — all logic in the browser
Real-time Sync	Not supported
Deployment	Open HTML file directly from desktop (<code>file://</code>)
Data Persistence	Cleared on browser cache reset, single device only

Limitations:

- Data not shared across devices
- Lost on browser cache clear
- No user authentication security (plain text passwords)
- No backend for push notifications or scheduled reminders

2. Architecture Transformation

The architecture was transformed from a single HTML file to a full-stack client-server application:

BEFORE: Browser ↔ localStorage

AFTER:

Browser (HTML/CSS/JS) → Node.js/Express API → Turso (Cloud SQLite)
Browser ← Socket.io (Real-time) ← Node.js/Express API

Layer	Technology	Purpose
Frontend	HTML5, CSS3, Vanilla JS	UI (same as original, with API integration)
Backend	Node.js + Express	REST API server on port 3000
Real-time	Socket.io (v4)	WebSocket events for cross-device sync
Database	Turso (SQLite-compatible)	Cloud-hosted, persistent, globally distributed
Hosting	Railway (SFO region)	Auto-scaling Node.js hosting
Auth	bcrypt + JWT	Password hashing + token-based sessions

3. Backend Server Structure

A new `server/` directory was created with the following structure:

```
server/
├─ index.js           # Express app + Socket.io + CORS + static files
├─ db.js              # Database client (Turso cloud / local SQLite fallback)
├─ auth.js            # JWT token generation & verification middleware
├─ .env               # Environment variables (local dev)
├─ .env.example       # Template for production
├─ package.json       # Dependencies
├─ routes/
│   ├─ auth.js        # POST /register, POST /login, GET+PUT /profile
│   ├─ medications.js  # CRUD + dose logging + real-time broadcast
│   ├─ sessions.js     # CRUD triage sessions + real-time broadcast
│   └─ notifications.js # POST /store-token (FCM push tokens)
└─ tabibak.html       # Frontend (copied here for deployment)
```

4. Database — From localStorage to Turso Cloud SQLite

Step 1 Replace `better-sqlite3` with `@libsql/client`

The synchronous SQLite driver was replaced with Turso's async client, which works with both local files and cloud databases.

Step 2 Same SQL Schema

All tables use standard SQLite syntax — no migration needed:

Table	Purpose
<code>users</code>	Username, bcrypt password hash, profile, FCM token
<code>sessions</code>	Triage assessment history with full results (JSON)
<code>medications</code>	Medication name, dose, form, urgency flag
<code>medication_doses</code>	Dose times per medication (one-to-many)
<code>medication_log</code>	Daily dose taken tracking

Step 3 Turso Cloud Database Created

Database URL: `libsql://tabibak-hashemelhe1o2827.aws-ap-south-1.turso.io`

Free tier: 500 databases, 9 GB storage, 1B row reads/month

Fallback behavior: When `TURSO_DB_URL` + `TURSO_DB_TOKEN` are not set, the app falls back to a local SQLite file — keeping local development working without changes.

5. Environment Configuration

Environment variables via `dotenv` manage all configuration:

Variable	Local	Production (Railway)
<code>PORT</code>	3000	Auto-assigned by Railway
<code>NODE_ENV</code>	development	production
<code>JWT_SECRET</code>	dev-secret	Random 256-bit secret
<code>FRONTEND_URL</code>	http://localhost:3000	Railway domain URL
<code>TURSO_DB_URL</code>	<i>not set</i>	libsql://...turso.io
<code>TURSO_DB_TOKEN</code>	<i>not set</i>	JWT auth token

6. Real-Time Synchronization with Socket.io

WebSocket events are emitted whenever data changes, enabling instant cross-device sync:

Event	Triggered When	What Clients Do
<code>medications:updated</code>	Medication created or deleted	Refetch & re-render meds list
<code>medications:dose-logged</code>	Dose marked as taken	Refetch meds + update progress
<code>sessions:created</code>	Triage session saved	Refetch & refresh history screen

Flow:

Phone: Add Medication → `POST /api/medications` → Server saves to Turso
Server emits `medications:updated` → `room:user:john`
Laptop (in `room:user:john`) receives event → auto-refetches data → updates UI
→ **No page refresh needed**

7. Frontend Changes

Only minimal changes were needed in `tabibak.html` :

Change	Before	After
<code>API_BASE</code>	<code>'http://localhost:3000'</code>	<code>window.location.origin</code>
Data Storage	<code>localStorage</code> for everything	API calls to backend; <code>localStorage</code> only for JWT token + taken doses
Auth	Plain text, in-browser only	bcrypt + JWT via API
Login	Check <code>STATE.users</code> locally	<code>POST /api/auth/login</code>
Register	Save to <code>localStorage</code>	<code>POST /api/auth/register</code>
Medications	Local array + <code>localStorage</code>	<code>POST/GET/DELETE /api/medications</code>
Sessions	Local array + <code>localStorage</code>	<code>POST/GET/DELETE /api/sessions</code>
Real-time	None	Socket.io client auto-connects on login

8. Security Improvements

Area	Before	After
Passwords	Plain text in localStorage	bcrypt hashed (10 rounds)
Auth Sessions	Username in localStorage	JWT with 7-day expiry
CORS	No CORS	Restricted to <code>FRONTEND_URL</code> origin
Database	localStorage (any browser dev tools)	Turso cloud (authenticated API)
Secret Management	Hardcoded in HTML	Environment variables via dotenv

9. Deployment Pipeline

- 1. GitHub** — Code pushed to `github.com/hashemelhelo2827/tabibak`
- 2. Turso** — Cloud database created via Turso web UI (free tier)
- 3. Railway** — Node.js service deployed from `server/` directory
- 4. Environment Variables** — Turso credentials, JWT secret, `FRONTEND_URL` set via Railway CLI
- 5. Auto-deploy** — `railway up` uploads and deploys directly

Deployment URL: <https://truthful-enchancement-production-2411.up.railway.app/tabibak.html>

Region: San Francisco, California (sfo)

Hosting Cost: Free tier (Railway + Turso)

10. API Endpoints Summary

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Create account (bcrypt hashed)
POST	/api/auth/login	No	Login, returns JWT
GET	/api/auth/profile	Yes	Get profile
PUT	/api/auth/profile	Yes	Update profile
GET	/api/medications	Yes	List user's medications
POST	/api/medications	Yes	Add medication
DELETE	/api/medications/:id	Yes	Delete medication
POST	/api/medications/log	Yes	Log dose as taken
GET	/api/sessions	Yes	List triage sessions
POST	/api/sessions	Yes	Save triage session
DELETE	/api/sessions/:id	Yes	Delete session
POST	/api/notifications/store-token	Yes	Save FCM push token

11. Files Created / Modified

File	Status	Purpose
server/index.js	New	Express server, Socket.io, CORS, dotenv
server/db.js	New	Turso/LibSQL database client + schema init
server/auth.js	New	JWT generation + verification middleware
server/routes/auth.js	New	Auth API endpoints (register, login, profile)
server/routes/medications.js	New	Medications CRUD + dose logging + Socket.io broadcast
server/routes/sessions.js	New	Sessions CRUD + Socket.io broadcast
server/routes/notifications.js	New	FCM push token storage
server/package.json	New	Dependencies + scripts
server/.env	New	Local development environment
server/.env.example	New	Production environment template
server/.gitignore	New	Ignore node_modules, .env, DB files
tabibak.html	Modified	API integration, Socket.io client, dynamic URL

12. How to Run Locally

```
cd server
npm install
npm start
# Open http://localhost:3000/tabibak.html
```

Without Turso credentials, the app uses a local SQLite file automatically.

13. How to Manage Deployment

```
cd server
railway logs          # View server logs
railway open          # Open Railway dashboard
railway redeploy      # Redeploy after changes
railway variable --set KEY=VALUE  # Set env vars
```